

Modern Cloud Computing

Lecture: SaaS and FaaS

Chongzhou Fang

Rochester Institute of Technology

Today's Topics

- SaaS
 - Basic Concepts
 - Design Principles
- FaaS
 - Basic Concepts
 - AWS Lambda: an example
 - FireCracker

SaaS

- One application must serve many tenants
- Tenants may share compute, storage, messaging, and control-plane services
- The provider must isolate tenants while preserving economies of scale

Core SaaS Concepts

Concept	Meaning
Tenant	A customer, organization, or group that consumes the SaaS application
SaaS identity	User identity enriched with tenant context
Tenant isolation	Controls that prevent one tenant from accessing another tenant's resources
Data partitioning	Strategy for storing and separating tenant data
Noisy neighbor	One tenant's workload degrades another tenant's experience
Tenant onboarding	Provisioning/configuring everything needed for a new tenant
Tenant-aware operations	Observability and operations through the lens of tenant and tier

Silo, Pool, and Bridge Models

Model	Resource pattern	Strength	Tradeoff
Silo	Dedicated resources per tenant	Strong isolation, compliance fit	Higher cost and operational overhead
Pool	Shared resources across tenants	Economies of scale, agility	Requires strong isolation controls
Bridge	Mix of silo and pool	Flexible per-service design	More architecture and operations complexity

Silo Model

A silo model gives each tenant dedicated resources.

Examples:

- Separate database per tenant
- Separate compute stack per tenant
- Separate VPC or account per tenant

Tenants still share:

- Identity and onboarding
- Deployment automation
- Monitoring and operations

Pool Model

A pool model shares resources across tenants.

Examples:

- Shared database with tenant IDs
- Shared compute cluster
- Shared queues and caches
- Shared application services

Benefits:

- Lower cost per tenant
- Better utilization

Challenge: isolation must be enforced by software, policy, and runtime mechanisms.

Bridge Model

Bridge model is common in real SaaS systems.

Some services are pooled, others are siloed.

Examples:

- Shared frontend and identity service
- Dedicated database for regulated tenants
- Pooled background workers for low-risk tasks
- Siloed analytics service for large enterprise tenants

Design is often driven by compliance, cost, tenant tier, workload intensity, and noisy-neighbor risk.

SaaS Identity

SaaS identity is a combination of:

- User identity
- Tenant identity
- Roles and permissions
- Tenant tier and policy context

Data Partitioning

Data partitioning determines how tenant data is stored.

Common choices:

- Separate database per tenant
- Shared tables with tenant ID
- Hybrid strategy based on tenant tier or compliance

Partitioning affects:

- Cost
- Isolation
- Query performance
- Backup and restore
- Migration and tenant offboarding

Data Partitioning Tradeoffs

Pattern	Isolation	Cost efficiency	Operational complexity
Database per tenant	High	Low	High
Schema/table per tenant	Medium-high	Medium	Medium
Shared table + tenant ID	Lower by default	High	Requires careful policy enforcement
Hybrid	Variable	Variable	Highest design complexity

Noisy Neighbor Problem

A noisy tenant places load on shared resources and degrades other tenants.

Examples:

- One tenant issues expensive queries
- One tenant floods an API endpoint
- One tenant's analytics job saturates shared storage

SaaS systems need strategies to detect, limit, and isolate tenant-level resource consumption.

Noisy Neighbor Mitigation

Typical mechanisms:

- Per-tenant quotas and rate limits
- Dedicated resources for high-tier tenants
- Queue isolation or priority scheduling

Tenant Tiers

SaaS systems often differentiate tenants by tier.

Examples:

- Free
- Standard
- Enterprise
- Regulated / dedicated

Tiers may affect:

- Resource limits
- Feature access
- SLA and support
- Data retention
- Isolation model
- Monitoring granularity
- Backup and restore policy

Tenant Onboarding

Tenant onboarding is the process of introducing a new tenant into the SaaS environment.

It may provision and configure:

- Tenant record and metadata
- Identity provider mappings
- Roles and policies
- Data partitions
- Billing plan and tier
- Monitoring tags and operational metadata

Automated Tenant Onboarding

SaaS depends on a frictionless onboarding experience.

Tenant signup

- Create tenant record
- Configure identity and admin user
- Apply tenant tier and policy
- Provision required resources
- Enable metering and observability
- Tenant becomes active

SaaS Design Principles

- There is no one-size-fits-all SaaS architecture
- Services should be decomposed based on tenant load and isolation needs
- Tenant resources must be isolated across the architecture
- The system should be designed for growth
- Tenant metrics should be instrumented and analyzed
- Tenant onboarding should be automated and repeatable

No One-Size-Fits-All SaaS Architecture

Important design factors include:

- Compliance requirements
- Tenant market segments
- Tenant tiers
- Cost efficiency goals
- Isolation requirements
- Noisy-neighbor risks
- Operational model

A SaaS system may use different tenancy models in different parts of the architecture.

Example:

A low-risk analytics service may use pooled storage, while a compliance-sensitive payment service may use siloed storage.

Decompose Services by Tenancy Profile

Service property	Design implication
High compliance sensitivity	Prefer stronger isolation or siloed resources
Heavy or unpredictable load	Consider isolation to reduce noisy-neighbor impact
Low-risk shared functionality	Pool resources for efficiency
Premium-tier feature	May justify siloed deployment
Tenant-specific performance requirements	Use tier-aware scaling and placement
Shared operational service	Keep pooled and centrally managed

Isolate Tenant Resources

Tenant isolation is a foundational SaaS requirement.

The system must prevent one tenant from accessing another tenant's resources.

Isolation should be enforced across multiple layers:

- Identity and authentication
- Authorization policies
- Application logic
- APIs
- Databases and storage
- Messaging systems

Design for Growth

SaaS systems are often built to support rapid growth.

A scalable SaaS architecture should support:

- Fast tenant onboarding
- Repeatable provisioning
- Automated configuration
- Centralized tenant management
- Minimal per-tenant operational overhead
- Elastic infrastructure consumption
- Tier-aware scaling policies

Goal: Adding more tenants should not require proportional increases in manual operations.

Tenant Metrics and Tenant-Aware Operations

Tenant-aware operations require metrics such as:

- Requests per tenant
- Latency per tenant
- Error rate per tenant
- Resource consumption per tenant
- Feature usage per tenant
- Cost impact per tenant
- Noisy-neighbor indicators

SaaS Techniques Summary

SaaS problem	Common technique
Many tenants share one product	Tenant-aware identity and metadata
Prevent cross-tenant access	Isolation policies at app, data, IAM, and network layers
Store tenant data efficiently	Silo, pool, bridge, and data partitioning patterns
Control noisy tenants	Quotas, throttling, workload isolation, tenant-aware scaling
Add tenants quickly	Automated onboarding and provisioning
Operate shared services	Tenant-aware logs, metrics, billing, and health views
Adopt SaaS in enterprises	Integration, SLA analysis, and hybrid-cloud governance

FaaS

What Is FaaS/Serverless?

Function as a Service runs application code as event-triggered functions.

The developer provides:

- Function code
- Runtime spec or container image
- Handler entry point
- Trigger configuration
- Memory, timeout, and permissions

The provider manages:

- Servers
- Runtime execution environments
- Scheduling and scaling
- Fault tolerance
- Invocation routing
- Logs, metrics, and billing

FaaS Programming Model

A function is usually defined by:

- Function identifier
- Language runtime
- Code package or image URI
- Handler method
- Memory limit
- Timeout
- Permissions
- Event trigger

The runtime invokes the handler when an event arrives.

FaaS: Typical Use Cases

Good fits:

- Request-response APIs
- Web and mobile backends
- File processing after object upload
- Queue/message processing
- Real-time stream processing
- Infrastructure automation
- Scheduled tasks
- Lightweight glue code between cloud services

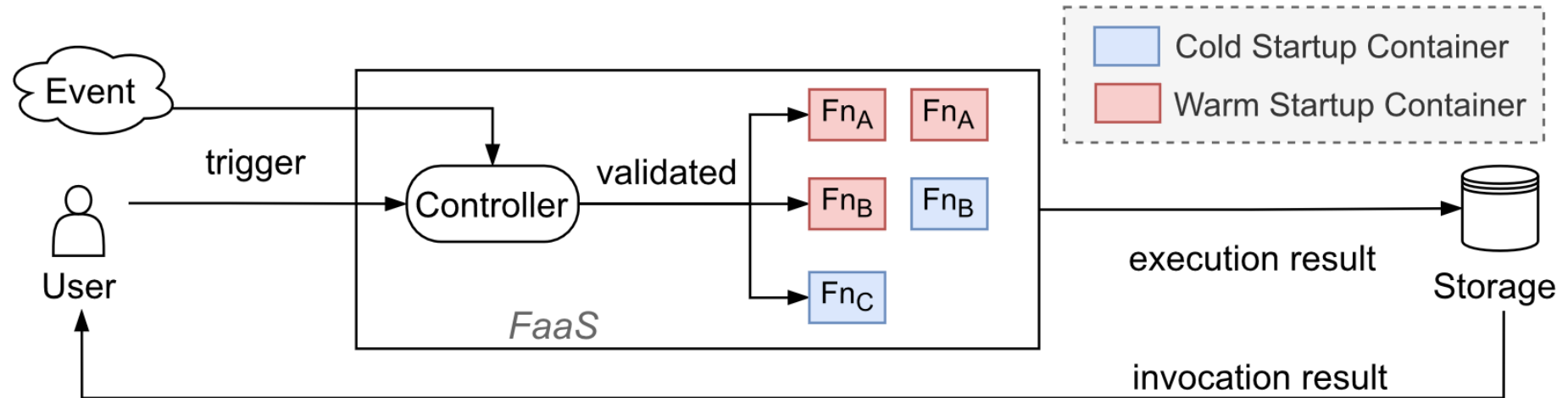
Less ideal:

- Long-running stateful services
- Applications requiring fine control over infrastructure

FaaS Features

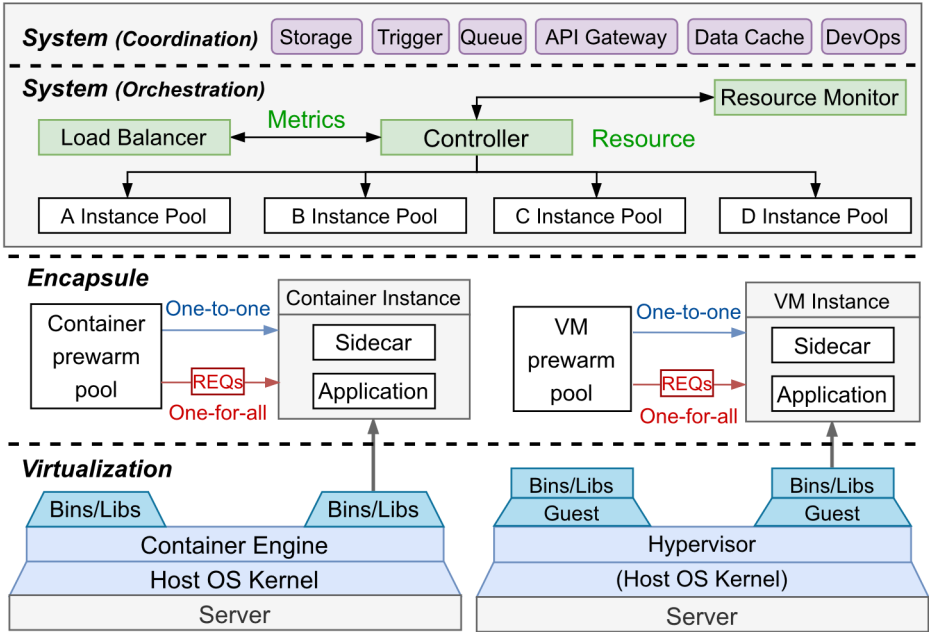
- Auto-scaling
- Flexible scheduling
- Event-driven execution
- Transparent development
- Pay-as-you-go billing

FaaS Invocation Flow



Serverless Architecture Layers

Layer	What it handles
Virtualization	Sandbox isolation for functions
Encapsule	Runtime, sidecars (middleware), triggers, and prewarm pools
System orchestration	Scheduling, scaling, load balancing, resource management
System coordination	BaaS (Backend-as-a-Service) services such as storage, queues, API gateway, DevOps tools



Virtualization Layer

The virtualization layer provides the sandbox where the function runs.

Options include:

- Traditional VM
- Linux container
- Secure container
- Unikernel
- MicroVM

The core tradeoff:

Strong isolation + compatibility + low startup latency + high density

Difficult to optimize simultaneously.

Virtualization Options

Option	Startup	Isolation	Typical issue
Traditional VM	Slow	Strong	Too heavy for fine-grained functions
Container	Fast	Weaker	Shared kernel increases attack surface
Unikernel	Fast	Potentially strong	Less general application support
MicroVM	Fast-medium	Strong	Requires specialized VMM design

Encapsule Layer

The encapsule layer packages function execution support around the sandbox.

It may include:

- Runtime wrapper
- Sidecar or shim process
- Metrics and logging collector
- Function metadata
- Trigger adapter
- Code/dependency loading logic

Purpose: decouple user function logic from platform management logic.

System Orchestration Layer

The orchestration layer decides how functions are placed and scaled.

It handles:

- Request routing
- Scheduling
- Load balancing
- Resource monitoring
- Cold-start mitigation
- Idle resource reclamation
- Scale-to-zero and scale-out

System Coordination Layer

FaaS applications are usually composed with managed backend services.

Examples:

- Object storage
- Databases
- Queues
- Streams
- Event buses
- API gateways
- Identity and secrets
- CI/CD and deployment tools

AWS Lambda: User View

AWS Lambda is a serverless compute service that runs code without provisioning or managing servers.

Lambda manages:

- Server maintenance
- Capacity provisioning
- Scaling
- Patching
- Execution environments
- Routing
- Fault tolerance

The developer focuses on application logic and event integration.

AWS Lambda Function Building Blocks

A Lambda function includes:

- Code package or container image
- Runtime
- Handler
- Memory configuration
- IAM execution role
- Environment variables
- Event sources and triggers
- Logs and telemetry

Events can come from API Gateway, S3, SQS, EventBridge, Kinesis, DynamoDB, and many other services.

Lambda Execution Environment

Lambda invokes a function inside a secure isolated execution environment.

The execution environment provides:

- Runtime process
- Function code and dependencies
- `/tmp` ephemeral storage
- Environment variables
- Credentials and permissions
- Lifecycle support for runtime and extensions

Lambda Lifecycle: Init, Invoke, Shutdown

Lambda execution environment phases:

1. Init

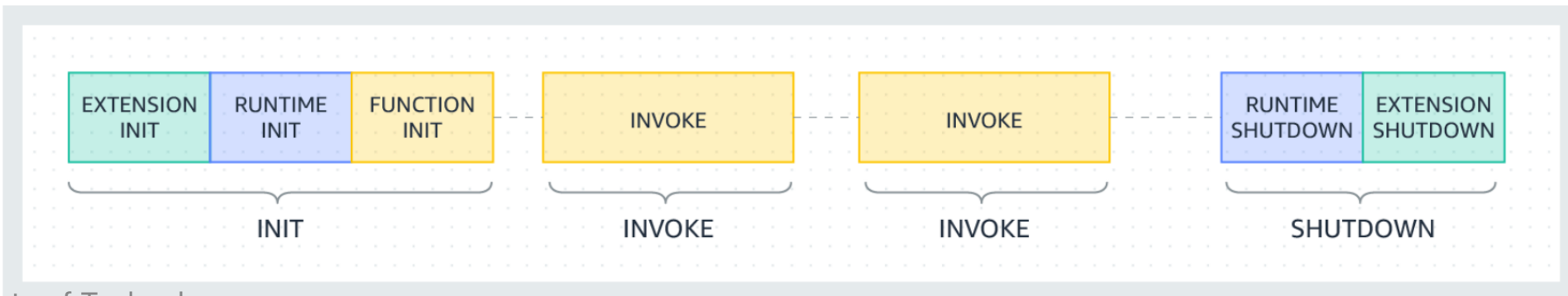
- Start extensions
- Bootstrap runtime
- Run function static initialization

2. Invoke

- Runtime receives event
- Handler runs
- Extensions participate in invocation

3. Shutdown

- Runtime and extensions clean up



Cold Starts and Warm Starts

A cold start occurs when Lambda must prepare a new execution environment.

Cold start work includes:

- Downloading code
- Setting up the environment
- Starting the runtime
- Running initialization code

A warm start reuses a previously initialized environment.

The environment can preserve initialized objects and `/tmp` contents, but applications should not assume persistence.

Reducing Cold Starts

Common mechanisms:

- Smaller deployment packages
- Prewarming
 - One-to-one
 - Predictive
 - One-for-all (template prewarming)

Provisioned concurrency pre-initializes execution environments so they are ready before invocation.

Lambda Concurrency

Lambda concurrency is the number of in-flight requests a function handles simultaneously.

Formula:

Concurrency = average requests per second × average request duration in seconds

If one request is already using an execution environment, another concurrent request needs another environment.

Lambda scales by provisioning more execution environment instances.

Reserved vs. Provisioned Concurrency

Control	Meaning	Use
Reserved concurrency	Minimum and maximum concurrency reserved for a function	Protect critical functions and cap runaway scaling
Provisioned concurrency	Pre-initialized environments allocated to a function	Reduce cold-start latency
Unreserved concurrency	Shared regional concurrency pool	Default behavior for most functions

Lambda Scaling Behavior

Lambda can scale each function independently.

AWS documents a maximum concurrency scaling rate of:

| 1,000 additional execution environment instances every 10 seconds per function per Region.

If requests arrive faster than a function can scale, or if concurrency limits are reached, requests may be throttled.

FaaS Behind the Curtain

The user sees:

```
upload function → configure trigger → function runs
```

Cloud provider does:

```
authenticate request  
→ route invocation  
→ locate or create execution environment  
→ load code/runtime  
→ invoke handler  
→ collect logs/metrics  
→ recycle or retain environment
```

Firecracker: Motivation

Serverless platforms need a difficult combination:

- Strong security isolation
- Low overhead
- High density
- Fast startup
- Software compatibility
- Consistent performance

Traditional tradeoff:

VMs: strong isolation, higher overhead

Containers: low overhead, weaker shared-kernel boundary

Firecracker was designed to avoid this tradeoff for serverless workloads.

Firecracker: What It Is

Firecracker is a lightweight Virtual Machine Monitor specialized for serverless and container workloads.

It uses KVM but replaces a general-purpose VMM such as QEMU with a minimal design.

Key idea:

- Provide VM-level isolation with container-like startup and density properties.

Firecracker is used in AWS Lambda.

Firecracker Design Goals

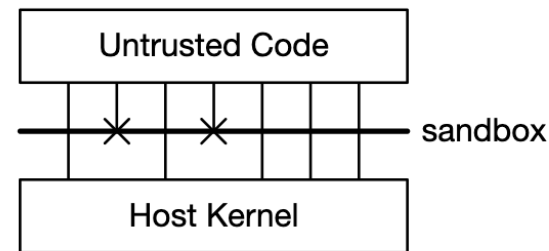
Firecracker targets:

- Isolation
- Overhead and density
- Performance
- Compatibility with unmodified Linux binaries
- Fast switching between workloads
- Soft allocation and oversubscription

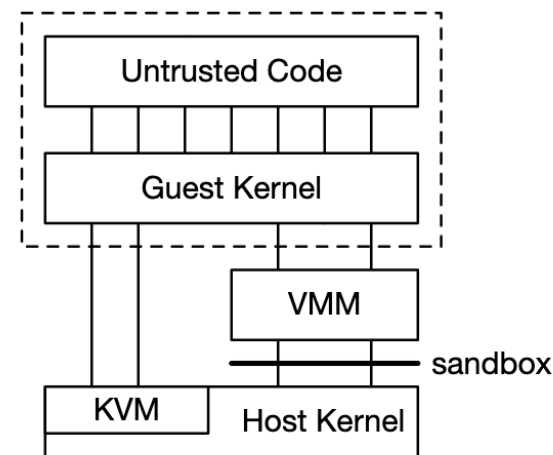
These goals come directly from the economics and security requirements of multi-tenant serverless compute.

Containers vs. Virtualization

Approach	Boundary	Strength	Weakness
Linux containers	Shared host kernel with namespaces/ cgroups/seccomp	High density and fast startup	Kernel attack surface and compatibility tradeoff
KVM virtualization	Guest kernel + VMM + host kernel	Stronger isolation and compatibility	Traditional VMMs can be heavy
Firecracker MicroVM	Minimal VMM + KVM + small guest	Strong isolation with low overhead	Specialized; not a general VM replacement



(a) Linux container model



(b) KVM virtualization model

What Firecracker Removes

Compared with general-purpose VM systems, Firecracker intentionally omits features not needed for serverless:

- BIOS
- Legacy device emulation
- PCI
- Arbitrary kernel boot support
- General desktop/server emulation features

Minimalism reduces attack surface and startup overhead.

Firecracker Performance Targets

Firecracker can provide:

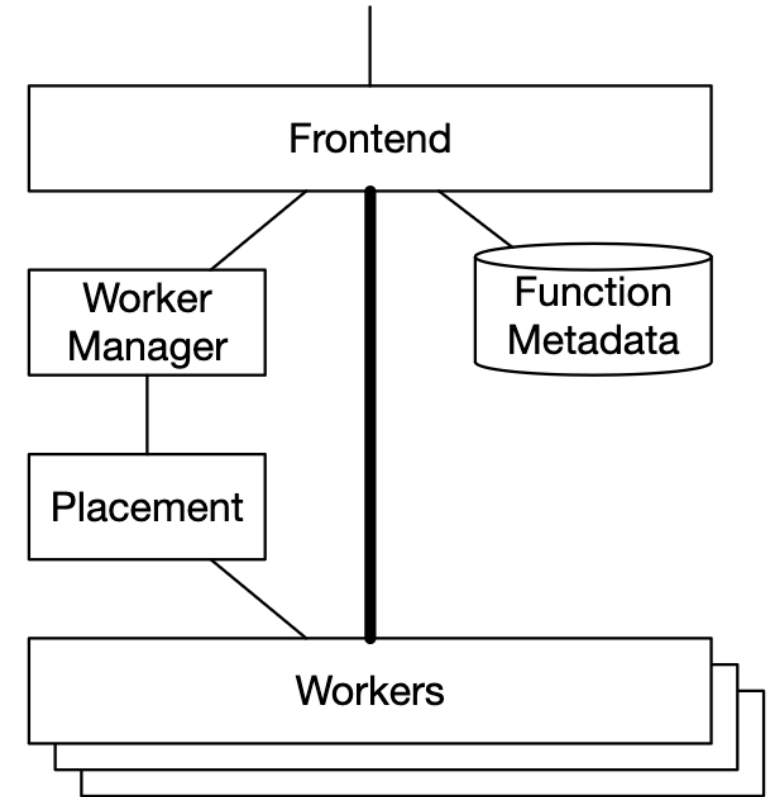
- Less than 5 MB memory overhead per container with minimal Linux guest configuration
- Boot to application code in less than 125 ms
- Up to 150 MicroVMs per second per host

Lambda Event Path Internals

The Firecracker paper describes a simplified AWS Lambda event path:

```
Invoke traffic arrives at frontend
→ request authenticated and authorized
→ function metadata loaded
→ Worker Manager performs sticky routing
→ worker slot selected or created
→ payload sent to worker
→ function executes in MicroVM slot
```

Sticky routing improves cache locality and amortizes code loading cost.



Lambda Worker Architecture

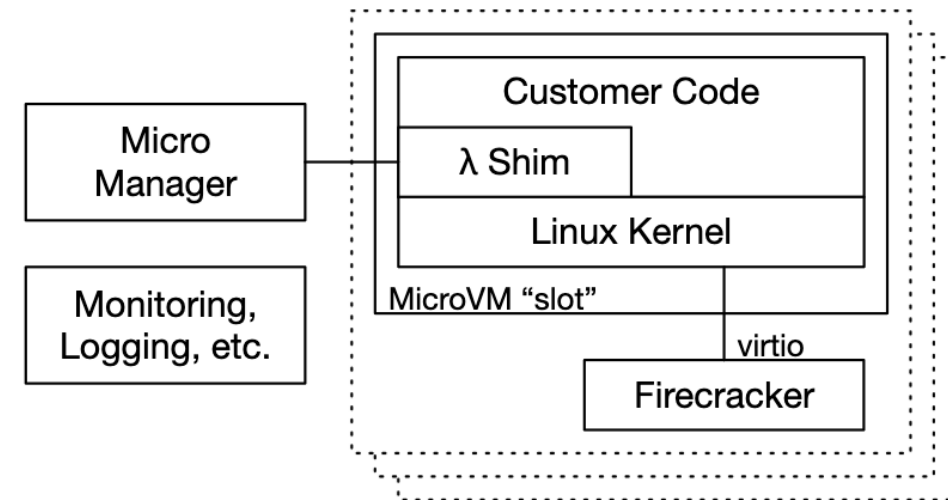
A Lambda worker (a server) has many MicroVM slots.

Each slot provides a pre-loaded execution environment for a single function.

Inside the slot:

- Customer code
- Lambda shim
- Minimal Linux kernel/userland
- Firecracker MicroVM boundary

Worker-side components manage monitoring, logging, slot control, and communication with the Lambda frontend.



Slot Reuse

A slot is used for:

- One function
- One concurrent invocation at a time
- Many serial invocations of that same function

```
first request → create/init slot  
later serial request → reuse same slot  
traffic spike → create additional slots  
idle period → slot eventually reclaimed
```

Placement Service

When no slot is available, Lambda's Worker Manager asks Placement to create one.

Placement considers:

- CPU
- Memory
- Network
- Storage
- Load distribution

MicroManager and Shim

In the Firecracker paper's Lambda worker design:

- A Firecracker process manages each MicroVM
- The Lambda shim runs inside the MicroVM
- MicroManager manages Firecracker processes on the worker
- Invocation payloads cross the MicroVM boundary through a network path

Slot State Transitions

A Lambda slot can move through states:

- Init
- Idle
- Busy
- Dead

Idle slots consume memory to preserve function state.

Busy slots consume CPU, memory bandwidth, caches, network, and storage.

Soft allocation and oversubscription make high utilization possible.

FaaS Takeaways

- FaaS is event-driven, horizontally scalable, and pay-per-use.
- Serverless is broader than FaaS because BaaS services are essential.
- Execution environments move through init, invoke, reuse, and shutdown.
- Cold starts are the visible symptom of behind-the-curtain sandbox creation.
- Firecracker shows how providers can specialize virtualization for serverless economics.
- FaaS hides servers, but does not remove scheduling, isolation, placement, or resource management.

References

- AWS Well-Architected Framework: SaaS Lens. <https://docs.aws.amazon.com/wellarchitected/latest/saas-lens/saas-lens.html>
- Seifert, Kühnel, Sackmann. "Hybrid Clouds Arising from Software as a Service Adoption." ACM Computing Surveys, 2023. DOI: 10.1145/3570156
- AWS Lambda Developer Guide. <https://docs.aws.amazon.com/lambda/latest/dg/>
- Li et al. "The Serverless Computing Survey: A Technical Primer for Design Architecture." ACM Computing Surveys, 2022. DOI: 10.1145/3508360
- Agache et al. "Firecracker: Lightweight Virtualization for Serverless Applications." NSDI 2020.